Halstead Metrics Calculation Matrix

Course: Software Engineering Laboratory (LAB 10)
Focus: Static Code Complexity & Cognitive Program Volume Analysis

1. Target Source Code Segment

Below is the target Python function selected for algorithmic complexity verification. It performs a classic linear scan, conditional filtering, and basic parameter state mapping over a dataset.

```
def calculate_average_score(scores):
    total = 0
    count = 0
    for score in scores:
        if score >= 50:
            total = total + score
            count = count + 1
    if count > 0:
        return total / count
    else:
        return 0
```

2. Primitive Token Identification Registry

Halstead metrics separate software source structures strictly into *Operators* (algorithmic actions, reserved punctuation, controls) and *Operands* (variables, literal constants, data labels).

Operator ID	Symbol / Keyword	Count (f■)
1	def (Function declaration)	1
2	() (Call / Parameter block)	2
3	= (Assignment operator)	4
4	for ... in (Iteration construct)	1
5	if (Conditional structural fork)	2
6	>= (Relational comparison)	1
7	+ (Arithmetic addition)	2
8	> (Relational comparison)	1
9	return (Control flow return)	2
10	/ (Arithmetic division)	1
11	else (Alternative branch)	1

-	**Distinct Operators (n■)**	**11**
-	**Total Operators (N■)**	**18**

Operand ID	Identifier / Literal Value	Count (f■)
1	calculate_average_score	1
2	scores	1
3	total	4
4	0	3
5	count	5
6	score	3
7	50	1
8	1	1
-	**Distinct Operands (n■)**	**8**
-	**Total Operands (N■)**	**19**

3. Halstead Metrics Evaluation Matrix

Using the primary tokens collected from the step above, the derived mathematical complexity metrics are calculated below:

Halstead Metric Metric	Mathematical Formula	Calculated Value	Technical Significance
Program Vocabulary (n)	n = n■ + n■	**11 + 8 = 19**	Total number of unique tokens in the programmatic ecosystem.
Program Length (N)	N = N■ + N■	**18 + 19 = 37**	Total cumulative occurrence of all operational and data tokens.
Estimated Length (N■)	N■ = n■·log■(n■) + n■·log■(n■)	**11·(3.459) + 8·(3.0) = 62.05**	Theoretical length prediction based purely on component vocabulary.
Program Volume (V)	V = N · log■(n)	**37 · log■(19) = 157.17 bits**	The physical processing information footprint required to host the code block.
Difficulty Level (D)	D = (n■ / 2) · (N■ / n■)	**(11 / 2) · (19 / 8) = 13.06**	Measures susceptibility to coding errors during implementation workflows.
Effort (E)	E = D · V	**13.06 · 157.17 = 2,053.04**	Mental processing effort required to translate design constraints to code.
Time to Implement (T)	T = E / 18	**2053.04 / 18 = 114.06 seconds**	Theoretical execution timeline mapped out in Stroud number parameters (18 items/sec).
Delivered Bugs (B)	B = V / 3000	**157.17 / 3000 = 0.052**	Estimated error count present in product release builds.

4. Key Metrics Synthesis

• **High Density Efficiency:** The code exhibits a tight core vocabulary profile ($n=19$), meaning code reusability across data markers ($score$, $total$, $count$) is maintained efficiently.

• **Low Defect Propensity:** A predicted Delivered Bugs metric of approximately 0.052 proves that single-responsibility functions keep cognitive complexity below risk baselines.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.